



**INSTITUTE FOR ADVANCED COMPUTING AND
SOFTWARE DEVELOPMENT, AKURDI, PUNE**

**"StayHub - PG Accommodation Booking
and Management System"**

PGCP-AC Feb 2026

Submitted By:

Group No: 8

Roll No.

Name of Student

262012

Anuj Jain

Ms. Divya Mishra

Mr. Anil Sharma

Project Guide

Centre Coordinator

ABSTRACT

StayHub - PG Accommodation Booking and Management System is an end-to-end digital accommodation portal designed to integrate student and working professional hostel discovery with automated room inventory management, online reservation workflows, and Razorpay payment integration. It allows PG property owners (landlords) to efficiently track and manage room inventories, available beds, property amenities, and tenant bookings while providing customers with a seamless, real-time online accommodation search and booking experience.

The platform leverages modern web development technologies: React 18 and Vite for a highly responsive, modern single-page application (SPA) frontend user interface, and Java 21 with Spring Boot 3.5 for a robust, enterprise-grade RESTful API backend architecture. Security is enforced through Spring Security with JSON Web Tokens (JWT) for stateless authentication and Role-Based Access Control (RBAC). Payments and rent reservations are securely processed via Razorpay API integration with automated HMAC SHA-256 signature verification.

StayHub addresses the operational challenges faced by hostel managers, PG owners, and tenants by replacing manual ledgers and unverified phone bookings with an integrated, multi-tenant digital solution. The platform features strict data governance, bed availability synchronization, soft-delete booking cancellations (status = CANCELLED), and transactional cascading cleanup (@Transactional) to ensure maximum database consistency and reliability.

ACKNOWLEDGEMENT

I take this occasion to thank God almighty for blessing us with His grace and taking our endeavor to a successful culmination. I extend my heartfelt thanks to our esteemed guide **Ms. Divya Mishra** for providing me with the right guidance and advice at the crucial juncture and showing me the right way.

I sincerely thank our respected Centre Co-Ordinator, **Mr. Anil Sharma**, for allowing me to use the available facilities at IACSD Akurdi, Pune. I would also like to thank the other faculty members at this occasion. Last but not least, I would like to thank my friends and family for the support and encouragement they have given me during our project work.

Anuj jain (260241220029)

TABLE OF CONTENTS

Sr. No	Description	Page No.
1	Introduction	1
2	SRS (Software Requirement Specification)	9
3	Diagrams	15
3.1	ER Diagram (Entity Relationship Diagram)	15
3.2	Use Case Diagram	16
3.3	Data Flow Diagram (Level 0 & Level 1)	17
3.4	Activity Diagram	18
3.5	Class Diagram	21
3.7	Sequence Diagram	23
4	Database Design	24
4.1	Design:	24
4.2	Tables:	25
5	Snapshots	27
5.1	Home Page & Property Discovery Interface	27
5.2	PG Listing Details & Room Selection	28

Sr. No	Description	Page No.
5.3	Bookings Status Filter Tabs & Audit	29
5.4	Categorized User Management & Password Reset	30
5.5	Amenity Preset Chips & Interactive Badges Manager	31
5.6	Razorpay Payment Processing & Verification	31
6	Conclusion	32
7	References	33

1. INTRODUCTION

StayHub: A Comprehensive PG Accommodation Booking and Property Management System is a modern enterprise web application designed to solve the challenges associated with hostel accommodation discovery, tenant onboarding, room bed inventory tracking, and rent reservation processing. In rapidly expanding urban educational hubs and IT corridors across Pune, Mumbai, Bangalore, and Hyderabad, thousands of migrating students and young working professionals struggle to find reliable, verified PG (Paying Guest) accommodations. Simultaneously, PG property owners (landlords) rely on outdated paper registers, unverified phone calls, and manual rent tracking methods, leading to double-booked rooms, revenue leakage, and unorganized record management.

StayHub provides a robust, multi-tenant digital SaaS platform that connects PG Owners, Tenants (Customers), and System Administrators under a unified, secure portal. The platform digitizes property listing registration, city-based search filtering, real-time bed inventory allocation, amenity assignment, room reservation requests, and Razorpay digital payment verification.

Built using a decoupled 3-Tier Client-Server Architecture, StayHub utilizes React 18 with Vite for a high-performance single-page user interface (SPA) on the frontend, and Java 21 with Spring Boot 3.5 for a lightweight, scalable REST API service layer on the backend. Data persistence is managed via MySQL 8.0 RDBMS using Hibernate ORM and Spring Data JPA repositories. Stateless authentication and role authorization are secured using Spring Security and JSON Web Tokens (JWT).

1.1 Purpose of the System

The primary purpose of StayHub is to replace manual hostel record-keeping with an automated, transparent, and user-friendly web portal. The system aims to:

- Provide real-time visibility into available beds across registered PG properties.
- Allow tenants to search, inspect room configurations, check rent rates, and submit booking requests online.

- Enable PG property owners to list properties, manage room sharing types, assign amenities, and track tenant bookings.
- Integrate Razorpay payment gateway for instant online rent reservations and signature-verified digital receipts.
- Provide Administrators with super-user capabilities for managing user categories, system oversight, and data cleanup.

1.2 Scope of the Project

The scope of the StayHub project encompasses the complete software development lifecycle (SDLC)â€”from requirements engineering, database design, backend API implementation, frontend UI assembly, to end-to-end user acceptance testing (UAT). The functional boundary includes:

- **User Account Management:** Registration, JWT authentication, BCrypt password encryption, profile updates, Admin password resets, and 3-tier user category isolation (PG Owners, Customers, System Admins).
- **Property & Inventory Management:** Property listing creation, city location filtering, room inventory configuration (single, double, triple sharing), bed allocation math, amenity assignment (WiFi, Mess, AC, CCTV), and photo gallery management.
- **Booking & Reservation Engine:** Room reservation requests, check-in date selection, stay duration calculation, real-time bed decrementation, status filtering (Confirmed, Pending Payment, Cancelled), and soft-delete cancellations with bed restoration (+1 bed).
- **Payment Processing:** Integration of Razorpay API for test payment order generation, test transaction cap enforcement (₹15,000 limit), HMAC SHA-256 signature verification, and automated status transitions to CONFIRMED.
- **Database Governance & Integrity:** Implementation of @Transactional cascading deletion logic across relational tables to eliminate MySQL foreign key constraint exceptions when deleting properties or user accounts.

1.3 Objectives of StayHub

1. **Automated Bed Inventory Tracking:** Eliminate manual bed count discrepancies by automatically updating available beds upon booking creation or cancellation.
2. **Multi-Criteria Property Search:** Allow tenants to filter properties by City, Gender Preference (Boys, Girls, Co-ed), and Price Range.
3. **Secure Stateless Authentication:** Implement Spring Security with JWT tokens and BCrypt password encoding to protect REST endpoints.
4. **Seamless Payment Gateway Integration:** Automate payment order creation and signature verification via Razorpay SDK.
5. **Owner Multi-Tenant Data Isolation:** Enforce strict data boundaries so PG owners access only their own properties, rooms, and customer bookings.
6. **Transactional Cascade Deletion:** Ensure database integrity by cascading deletions across payments, bookings, rooms, amenities, and gallery images.
7. **Audit-Friendly Soft Deletion:** Retain historical booking data by updating cancelled reservations to status = CANCELLED instead of hard deleting records.
8. **Interactive UI Management:** Provide 1-click preset amenity chips and interactive pill badges for quick amenity assignment and deletion.
9. **Responsive Glassmorphism UI:** Deliver a modern, visually stunning React user interface built with CSS design tokens and smooth micro-animations.
10. **Standardized API Architecture:** Document all REST endpoints using OpenAPI 3.0 / Swagger UI for clean frontend-backend separation.

1.4 Functionalities Provided by StayHub

StayHub delivers comprehensive capabilities tailored for three distinct user roles: Administrators, PG Owners, and Customers.

1. Customer Functionalities:

- **User Registration & Login:** Register account with email, name, phone, and BCrypt-hashed password; receive JWT auth token upon login.

- **Property Search & Discovery:** Search PG listings by city or location; filter by sharing type and monthly rent rate.
- **Room Inspection:** View detailed property cards, total beds vs available beds, sharing types, and assigned amenities.
- **Room Booking Request:** Select room, specify check-in date and stay duration, and submit booking request.
- **Online Payment:** Pay room reservation deposit via integrated Razorpay gateway supporting UPI, Netbanking, and Cards.
- **Booking Status Tracking:** Filter personal reservations by status (Confirmed, Pending Payment, Cancelled) and view payment receipts.
- **Stay Cancellation:** Cancel pending or confirmed bookings, triggering soft-delete status update (CANCELLED) and restoring bed availability (+1 bed).

2. PG Owner Functionalities:

- **Property Registration:** Register PG properties with name, city, complete address, pincode, and description.
- **Room Inventory Setup:** Add rooms specifying room number, sharing type (Single, Double, Triple), total beds, available beds, and monthly rent.
- **Amenity Management:** Assign amenities to PG properties using 1-click preset chips (WiFi, Mess, AC, Laundry, CCTV, Parking, Gym) and manage assigned badges with instant removal buttons.
- **Gallery Photo Upload:** Upload high-resolution property photos categorized strictly by property ownership.
- **Tenant Booking Oversight:** View and track customer room bookings submitted for owned PG properties.

3. System Administrator Functionalities:

- **Categorized User Management:** View registered accounts separated into 3 quick-switch tabs: PG Owners, Customers, and System Admins.
- **Admin Password Reset:** Optional reset of user passwords with BCrypt re-encoding for account recovery.

- **Cascading User Deletion:** Delete user accounts with transactional cascading cleanup across bookings, payments, rooms, amenities, images, and PG listings.
- **System-Wide Audit & Control:** Full super-user access to inspect, update, or delete any property, room, booking, or payment record across the platform.

1.5 Role-Based Access Control (RBAC) Security Architecture

StayHub enforces strict Role-Based Access Control (RBAC) at both backend REST controller endpoints (using Spring Security annotations) and frontend navigation routes (using React Auth Context). The security matrix is detailed below:

Resource / REST Endpoint	HTTP Method	CUSTOMER	OWNER	ADMIN
/api/auth/register, /api/auth/login	POST	Public	Public	Public
/api/pg-listings (Search & View)	GET	Allowed	Allowed	Allowed
/api/pg-listings (Create / Update)	POST / PUT	Denied	Owned Only	Full Access
/api/pg-listings/{id} (Delete)	DELETE	Denied	Owned Only	Full Access
/api/rooms (Add Room / View)	POST / GET	View Only	Owned Only	Full Access
/api/amenities (Assign / Delete)	POST / DELETE	Denied	Owned Only	Full Access
/api/bookings (Create Reservation)	POST	Allowed	Denied	Full Access
/api/bookings/my-bookings	GET	Own Only	Owned PGs	Full Access

Resource / REST Endpoint	HTTP Method	CUSTOMER	OWNER	ADMIN
/api/bookings/{id} (Cancel Booking)	DELETE	Own Only	Owned PGs	Full Access
/api/payments/create-order	POST	Allowed	Denied	Full Access
/api/payments/verify	POST	Allowed	Denied	Full Access
/api/users (Categorized List / Delete)	GET / DELETE	Denied	Denied	Full Access

2. SOFTWARE REQUIREMENT SPECIFICATION (SRS)

The Software Requirement Specification (SRS) details the complete functional and non-functional requirements governing the design, implementation, and verification of StayHub.

2.1 Functional Requirements

- **1 User Registration:** The system shall allow new users to register an account by providing first name, last name, email, phone number, password, and role selection (CUSTOMER or OWNER). Passwords shall be encrypted using BCrypt password encoder before database insertion.
- **2 JWT Authentication:** The system shall authenticate users during login using registered email and password credentials, returning a signed JSON Web Token (JWT) containing user email and role payload for stateless session authorization.
- **3 Categorized User Display:** The system shall provide Administrators with a user management interface divided into 3 distinct tabs: PG Owners, Customers, and System Admins.
- **4 Admin Password Update:** The system shall allow Administrators to update user account details, including an optional new password field. If provided, the new password shall be BCrypt-encoded before persistence.
- **5 Transactional Cascade User Deletion:** The system shall execute @Transactional cascading cleanup when an Administrator deletes a user account, automatically removing associated payments, bookings, rooms, amenities, listing images, and PG properties without foreign key constraint errors.
- **6 PG Property Registration:** The system shall allow registered PG Owners to create new PG listings by providing PG name, city, full address, state, pincode, description, and target tenant type (Boys, Girls, Co-ed).
- **7 Property Search & Filter:** The system shall allow all users to search registered PG properties by city and filter results by monthly rent range or tenant type.

- **8 Room Inventory Management:** The system shall allow PG Owners to add rooms to owned PG listings, specifying room number, sharing type (Single, Double, Triple), total beds, available beds, and price per month.
- **9 Amenity Assignment Engine:** The system shall enable PG Owners to assign amenities using 1-click preset chips (WiFi, Mess, AC, Laundry, CCTV, Power Backup, Gym, Parking) and remove assigned amenity badges with instant deletion buttons.
- **10 Gallery Image Upload:** The system shall allow PG Owners to upload property photos scoped strictly to their owned PG listings.
- **11 Customer Room Booking:** The system shall allow authenticated Customers to select an available room, specify check-in date and stay duration, and submit a booking request.
- **12 Bed Inventory Decrementation:** The system shall automatically decrement the target room's available beds by 1 upon successful creation of a booking request.
- **13 Razorpay Payment Order Creation:** The system shall integrate Razorpay API to generate payment orders for pending bookings, enforcing a test transaction cap limit of ₹15,000.
- **14 Razorpay Signature Verification:** The system shall verify HMAC SHA-256 payment signatures returned by the Razorpay checkout modal before marking payment transactions as SUCCESS and updating booking status to CONFIRMED.
- **15 Soft-Delete Booking Cancellation:** The system shall support soft-deletion of booking reservations when a customer or owner clicks Cancel, updating booking status to CANCELLED and incrementing room available beds by +1 while retaining audit history.

2.2 Non-Functional Requirements

- **1 Performance & Response Time:** REST API endpoints shall process and respond to client requests within 200 milliseconds under standard operating loads. Database queries shall execute within 50 milliseconds using HikariCP connection pooling.

- **2 System Availability & Uptime:** The backend services shall maintain 99.9% uptime over a 12-month period. Unhandled exceptions shall be intercepted by a Global Exception Handler returning standardized JSON error payloads.
- **3 Data Security & Encryption:** Sensitive data, including passwords, shall be hashed using BCrypt with a cost factor of 10. JWT tokens shall expire after 24 hours. API requests shall enforce HTTPS transmission.
- **4 Scalability & Modular Design:** Backend source code shall follow a clean 3-tier modular architecture (Controller, Service, Repository, Entity, DTO). Database design shall comply with Third Normal Form (3NF).
- **5 Frontend Usability & Responsiveness:** The React user interface shall render fluidly across Mobile (375px+), Tablet (768px+), and Desktop (1280px+) screen resolutions, featuring modern glassmorphism styling, loading spinners, and toast feedback notifications.

2.3 Hardware and Software Interfaces

1. Hardware Configuration (Backend Server):

- **Processor:** Intel Core i5 / i7 or AMD Ryzen 5 / 7 (Quad-Core 2.5GHz+)
- **System Memory (RAM):** 8 GB DDR4 RAM (16 GB Recommended)
- **Hard Disk Storage:** 10 GB Solid State Drive (SSD) free space
- **Network Adapter:** Gigabit Ethernet / Wi-Fi Adapter

2. Hardware Configuration (Frontend Client):

- **Processor:** Dual-Core 1.8GHz or higher
- **System Memory (RAM):** 4 GB RAM
- **Display Resolution:** 1280 x 720 minimum (1920 x 1080 Recommended)
- **Input Devices:** Standard Keyboard and Mouse / Touchscreen

3. Software Interfaces & Environment Specs:

- **Operating System:** Windows 10 / 11, Linux (Ubuntu 22.04 LTS), or macOS

- **Programming Language:** Java 21 (JDK 21 LTS)
- **Backend Framework:** Spring Boot 3.5.14, Spring Data JPA, Spring Security
- **Database Management System:** MySQL 8.0 Community Server (InnoDB Engine)
- **Frontend Framework & Build System:** React 18, Vite 6, JavaScript (ES6+), Axios
- **Payment Gateway SDK:** Razorpay Java SDK 1.4.3 & Razorpay JS Checkout Modal
- **API Testing & Documentation Tools:** Swagger UI / OpenAPI 3.0, Postman

3. SYSTEM DIAGRAMS & ARCHITECTURE

This chapter presents the architectural and unified modeling language (UML) diagrams governing the StayHub system.

3.1 ER Diagram (Entity Relationship Diagram)

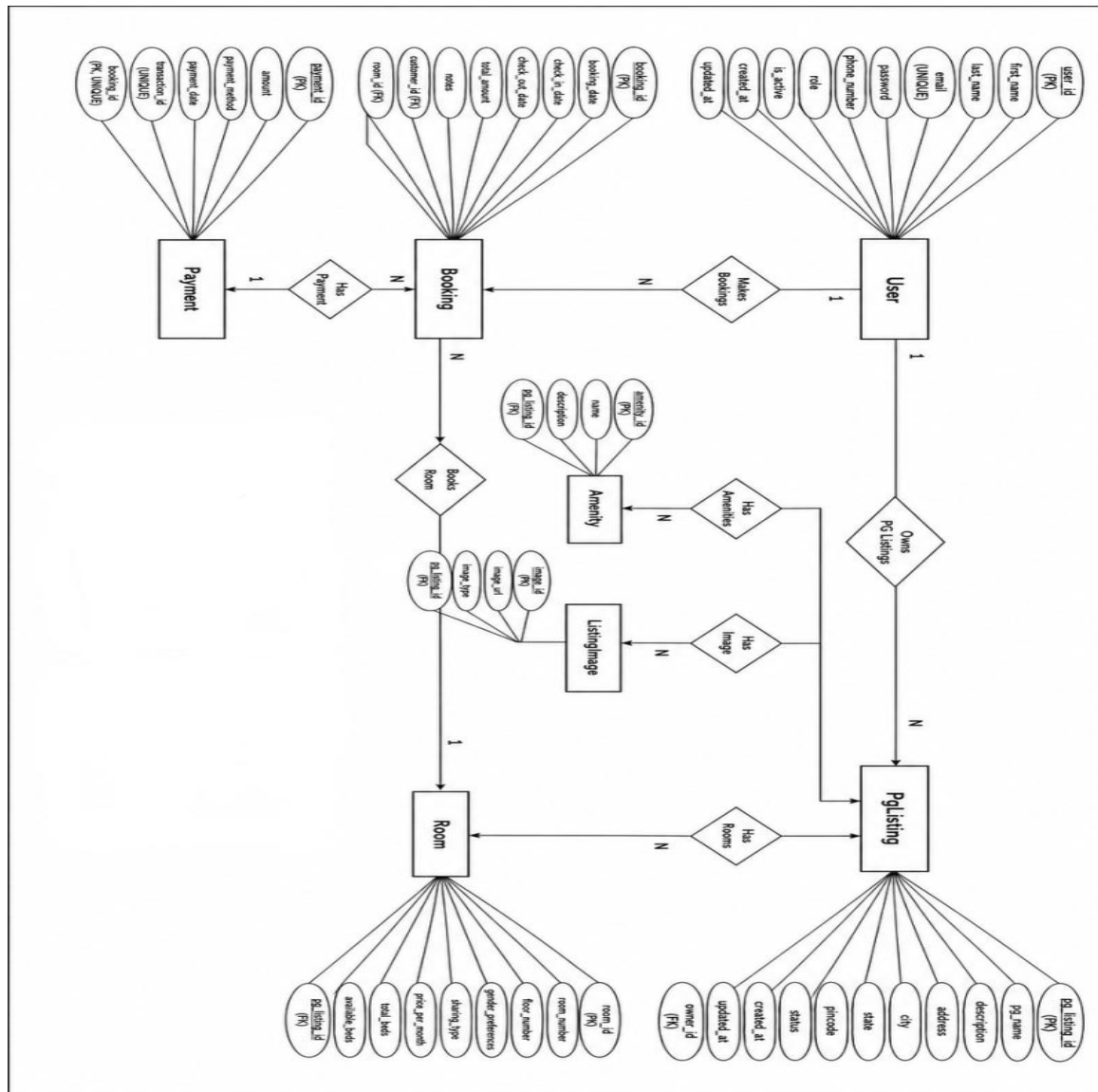


Figure 3.1: StayHub Application - Classic ER Diagram (Chen Notation) with Unidirectional JPA Architecture

Key entity associations depicted in Figure 3.1 include:

- **User to PgListing (1 : N):** User owns PG listings via `owner_id` foreign key reference.
- **PgListing to Room (1 : N):** PG listing has rooms via `pg_listing_id` foreign key reference.
- **PgListing to Amenity (1 : N):** PG listing has amenities via `pg_listing_id` foreign key reference.
- **PgListing to ListingImage (1 : N):** PG listing has images via `pg_listing_id` foreign key reference.
- **User to Booking (1 : N):** User makes bookings via `customer_id` foreign key reference.
- **Room to Booking (1 : N):** Booking reserves a room via `room_id` foreign key reference.
- **Booking to Payment (1 : 1):** Booking has a payment transaction via `booking_id` foreign key (UNIQUE).

3.2 Use Case Diagram

The Use Case Diagram defines interactions between primary system actors (Customer, PG Owner, System Admin) and external payment gateways (Razorpay).

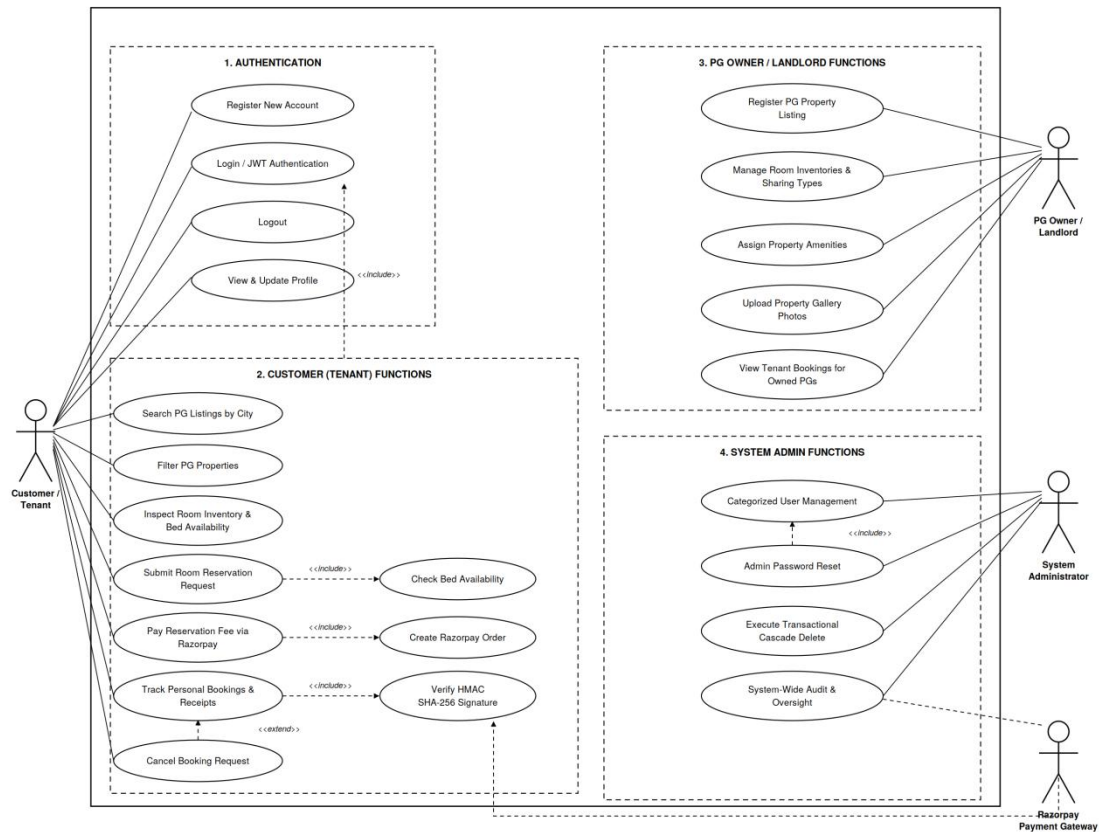


Figure 3.2: StayHub Application - UML Use Case Diagram

3.3 Data Flow Diagram (DFD Level 0 & Level 1)

The Data Flow Diagrams (DFDs) detail the flow of data across external entities, processes, and relational data stores in StayHub.

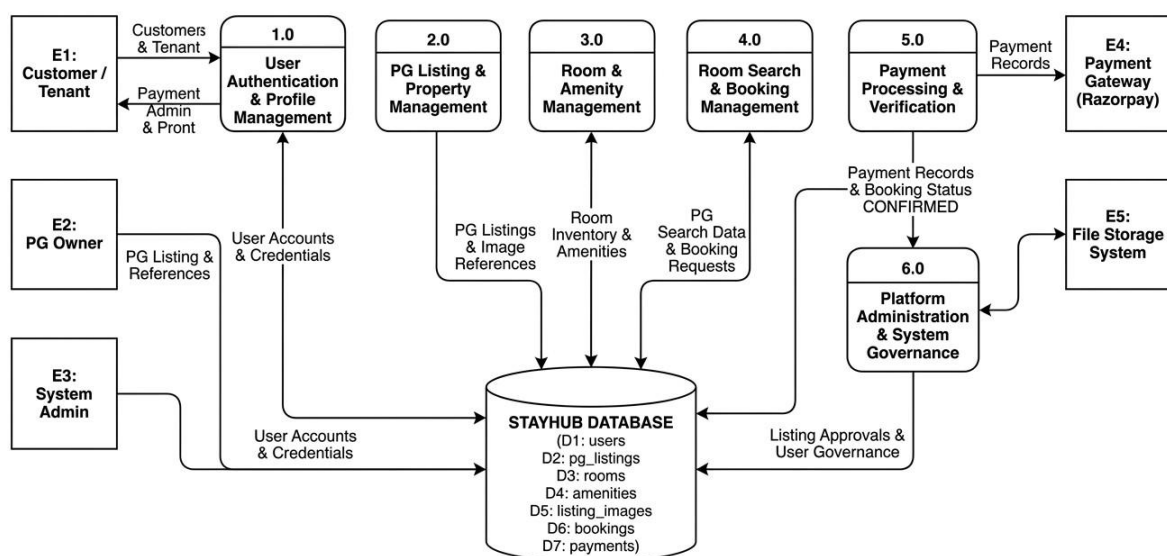
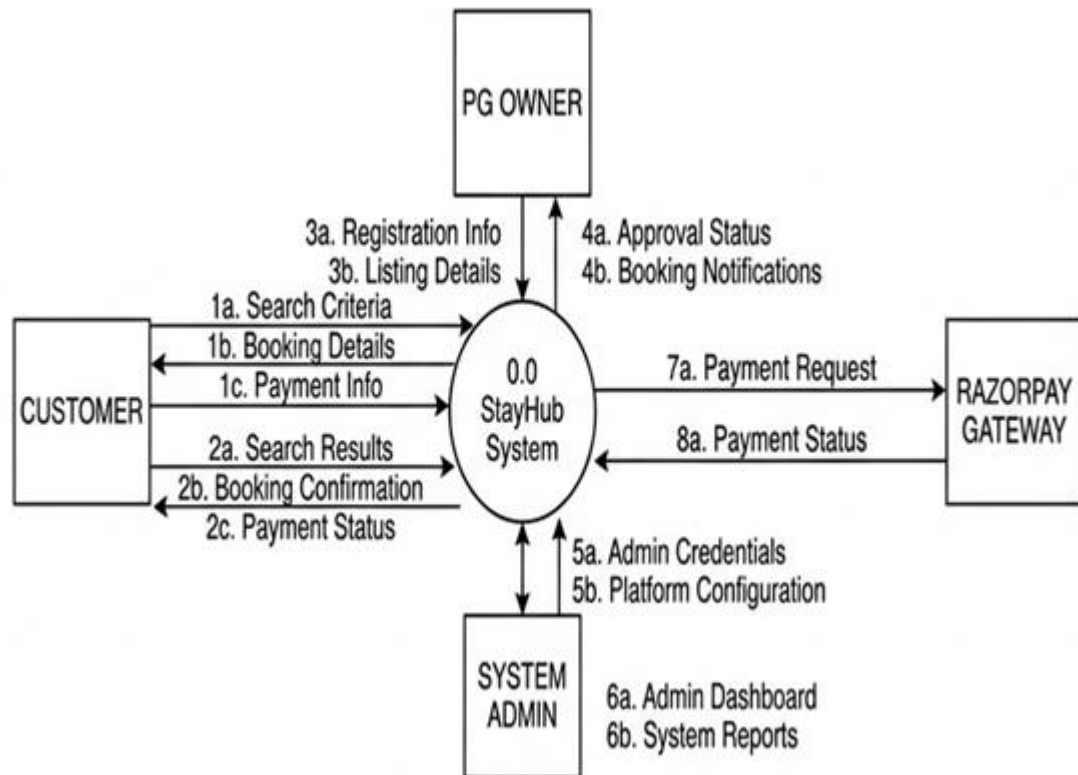
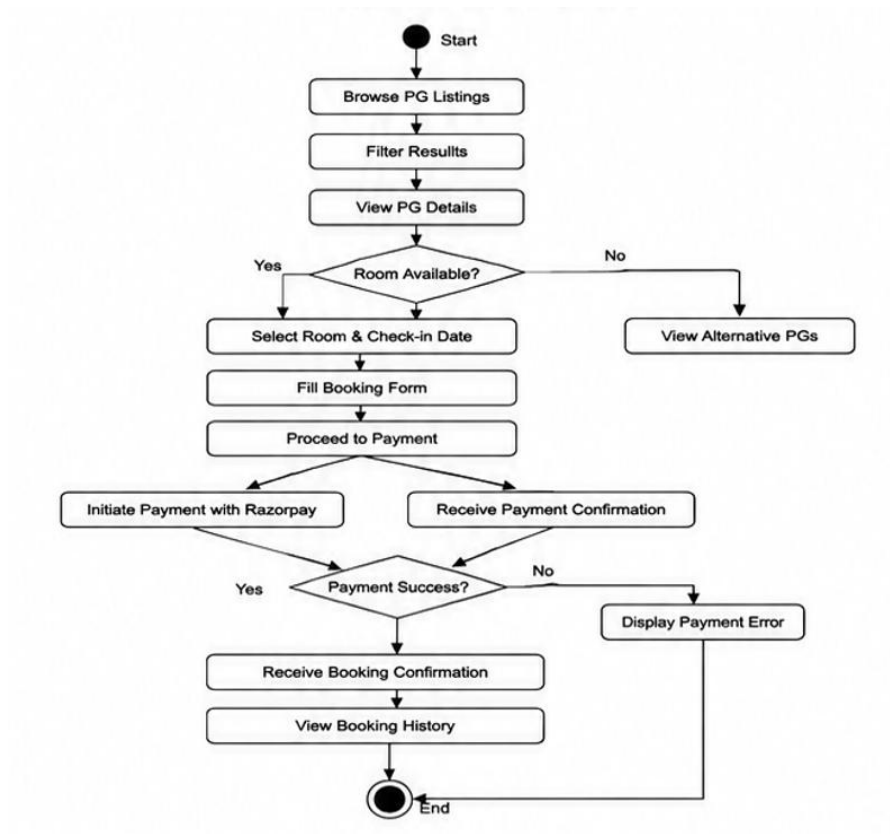


Figure 3.3: StayHub Application - Data Flow Diagram (DFD Level 0 & Level 1)

3.4 Activity Diagram



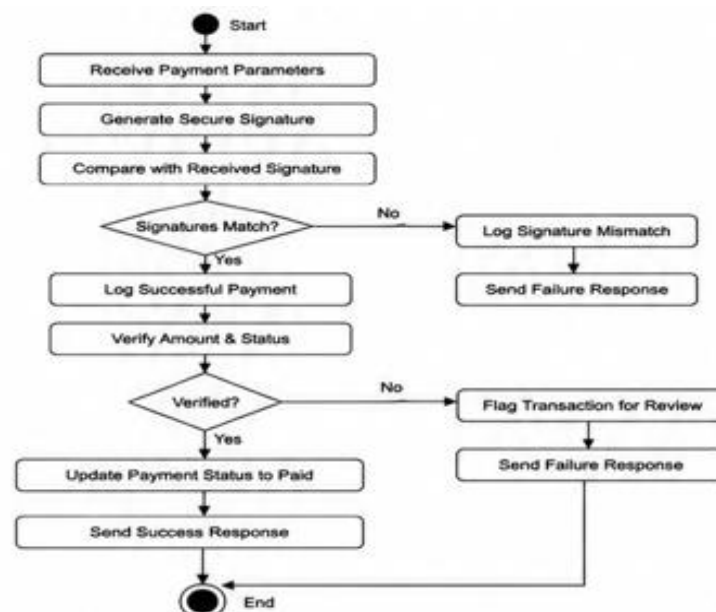
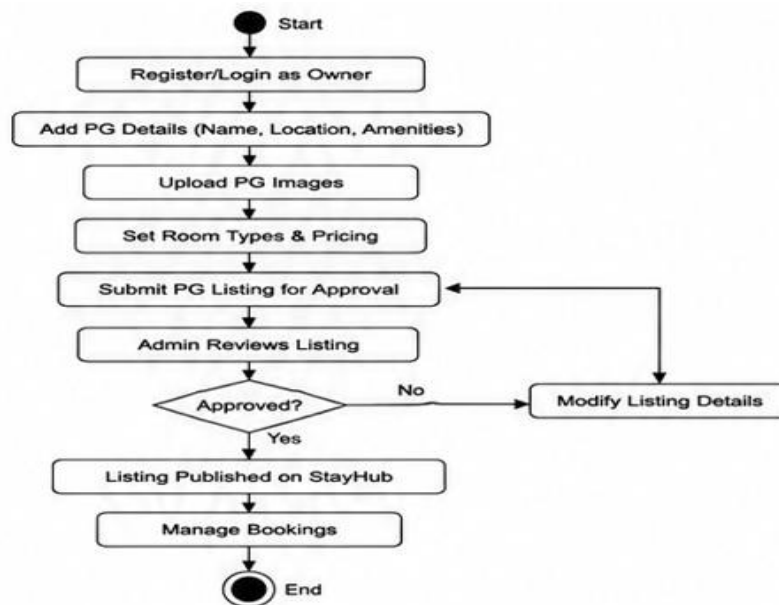


Figure 3.4: StayHub Application - UML Activity Diagrams

3.5 Class Diagram

StayHub

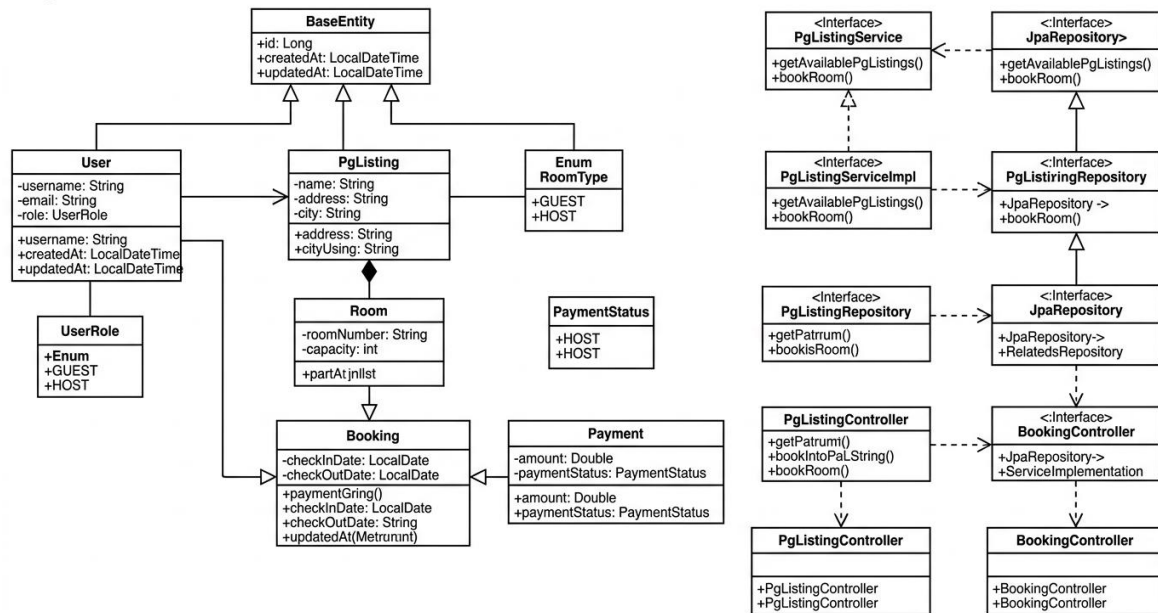


Figure 3.5: StayHub Application - UML Class Diagram

3.6 Sequence Diagram

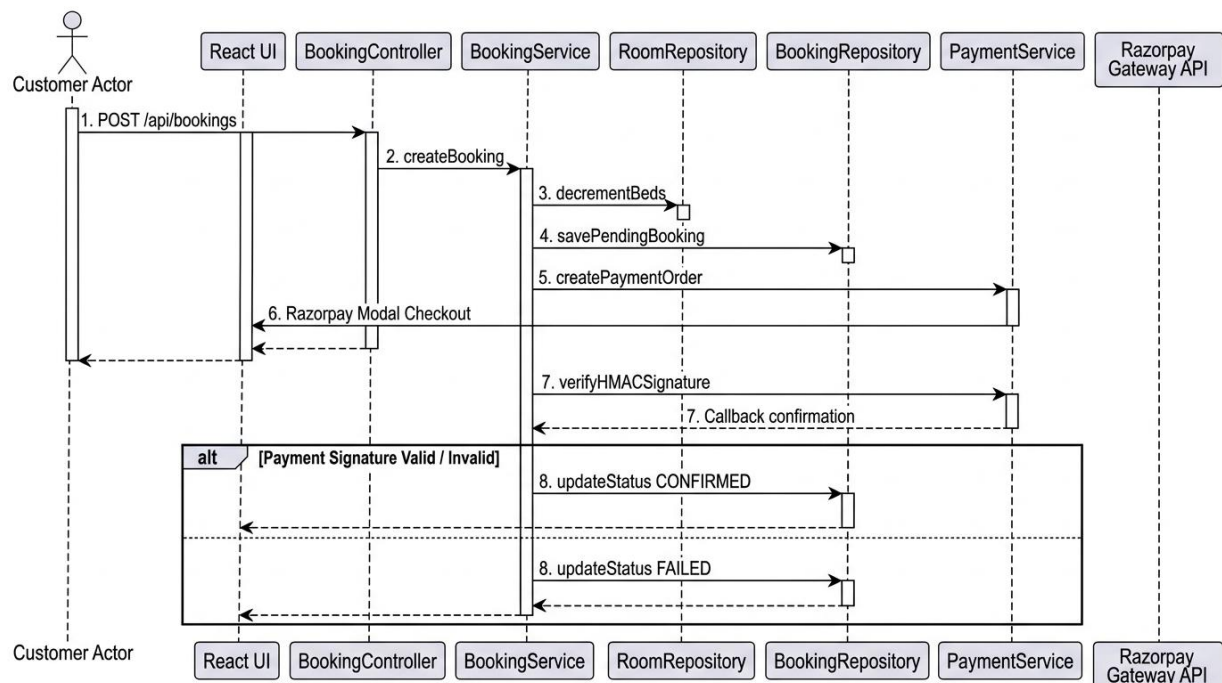


Figure 3.6: StayHub Application - UML Sequence Diagram

4. DATABASE DESIGN

The database layer is implemented using MySQL 8.0 RDBMS with InnoDB storage engine to guarantee ACID transaction compliance, foreign key constraint enforcement, and high concurrency via HikariCP connection pooling.

4.1 Design:

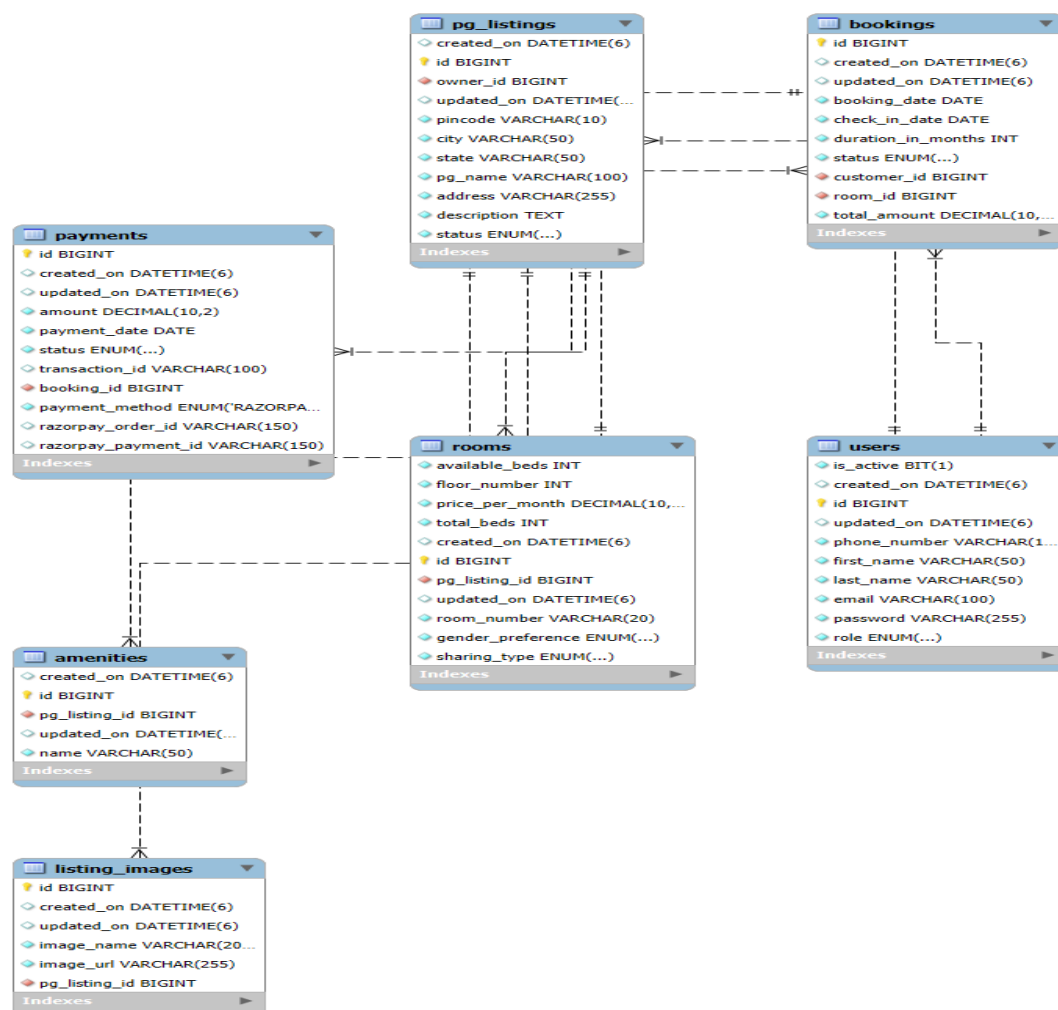


Figure 4.1: DataBase Design Diagram

4.2 Tables:

The following table structures depict the database design across all 7 normalized relational tables in Third Normal Form (3NF):

```
mysql> desc users;
```

Field	Type	Null	Key	Default	Extra
is_active	bit(1)	NO		NULL	
created_on	datetime(6)	YES		NULL	
id	bigint	NO	PRI	NULL	auto_increment
updated_on	datetime(6)	YES		NULL	
phone_number	varchar(15)	NO	UNI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	
role	enum('ADMIN', 'CUSTOMER', 'OWNER')	NO		NULL	

10 rows in set (0.00 sec)

Table 1: users

```
mysql> desc pg_listings;
```

Field	Type	Null	Key	Default	Extra
created_on	datetime(6)	YES		NULL	
id	bigint	NO	PRI	NULL	auto_increment
owner_id	bigint	NO	MUL	NULL	
updated_on	datetime(6)	YES		NULL	
pincode	varchar(10)	NO		NULL	
city	varchar(50)	NO		NULL	
state	varchar(50)	NO		NULL	
pg_name	varchar(100)	NO		NULL	
address	varchar(255)	NO		NULL	
description	text	NO		NULL	
status	enum('ACTIVE', 'INACTIVE')	NO		NULL	

11 rows in set (0.00 sec)

Table 2: pg_listings

```
mysql> desc rooms;
```

Field	Type	Null	Key	Default	Extra
available_beds	int	NO		NULL	
floor_number	int	NO		NULL	
price_per_month	decimal(10,2)	NO		NULL	
total_beds	int	NO		NULL	
created_on	datetime(6)	YES		NULL	
id	bigint	NO	PRI	NULL	auto_increment
pg_listing_id	bigint	NO	MUL	NULL	
updated_on	datetime(6)	YES		NULL	
room_number	varchar(20)	NO	MUL	NULL	
gender_preference	enum('FEMALE', 'MALE', 'UNISEX')	NO		NULL	
sharing_type	enum('DOUBLE', 'FOUR_SHARING', 'SINGLE', 'TRIPLE')	NO		NULL	

11 rows in set (0.00 sec)

Table 3: rooms

```
mysql> desc bookings;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
created_on	datetime(6)	YES		NULL	
updated_on	datetime(6)	YES		NULL	
booking_date	date	NO		NULL	
check_in_date	date	NO		NULL	
duration_in_months	int	NO		NULL	
status	enum('CANCELLED','CHECKED_IN','CHECKED_OUT','CONFIRMED','PENDING')	NO		NULL	
customer_id	bigint	NO	MUL	NULL	
room_id	bigint	NO	MUL	NULL	
total_amount	decimal(10,2)	NO		NULL	

10 rows in set (0.00 sec)

Table 4: bookings

```
mysql> desc payments;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
created_on	datetime(6)	YES		NULL	
updated_on	datetime(6)	YES		NULL	
amount	decimal(10,2)	NO		NULL	
payment_date	date	NO		NULL	
status	enum('FAILED','PENDING','REFUNDED','SUCCESS')	NO		NULL	
transaction_id	varchar(100)	YES		NULL	
booking_id	bigint	NO	UNI	NULL	
payment_method	enum('RAZORPAY')	NO		NULL	
razorpay_order_id	varchar(150)	YES		NULL	
razorpay_payment_id	varchar(150)	YES		NULL	

11 rows in set (0.00 sec)

Table 5: payments

```
mysql> desc amenities;
```

Field	Type	Null	Key	Default	Extra
created_on	datetime(6)	YES		NULL	
id	bigint	NO	PRI	NULL	auto_increment
pg_listing_id	bigint	NO	MUL	NULL	
updated_on	datetime(6)	YES		NULL	
name	varchar(50)	NO		NULL	

5 rows in set (0.00 sec)

Table 6: amenities

```
mysql> desc listing_images;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | bigint | NO   | PRI | NULL    | auto_increment |
| created_on | datetime(6) | YES | | NULL    | |
| updated_on | datetime(6) | YES | | NULL    | |
| image_name | varchar(200) | NO | | NULL    | |
| image_url | varchar(255) | NO | | NULL    | |
| pg_listing_id | bigint | NO | MUL | NULL    | |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Table 7: listing_images

5. SNAPSHOTS

5.1 Home Page & Property Discovery Interface

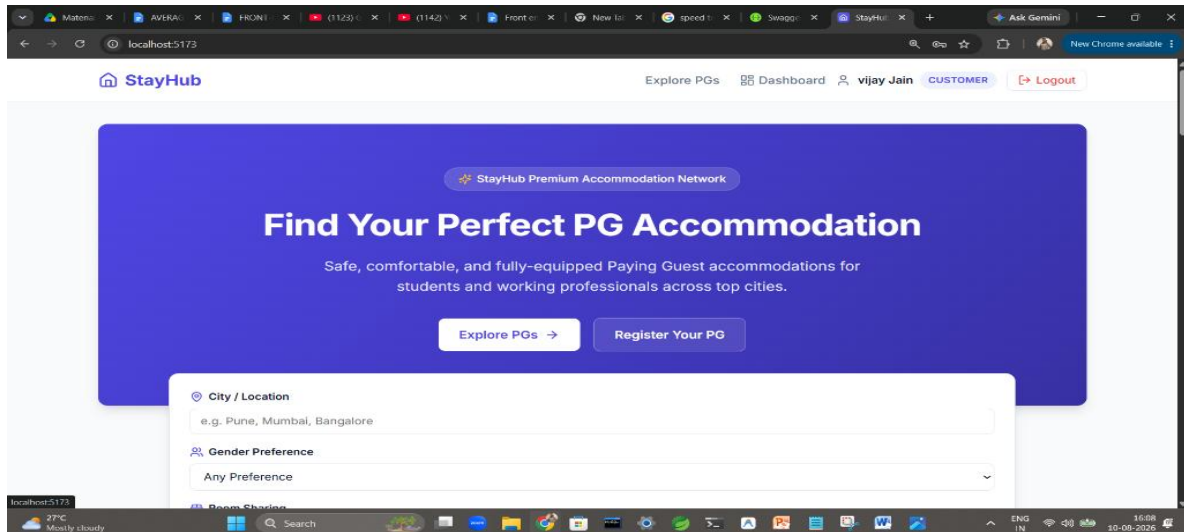


Figure 5.1: StayHub Home Page featuring live property discovery, city filtering, and gender preferences.

5.2 PG Listing Details & Room Selection

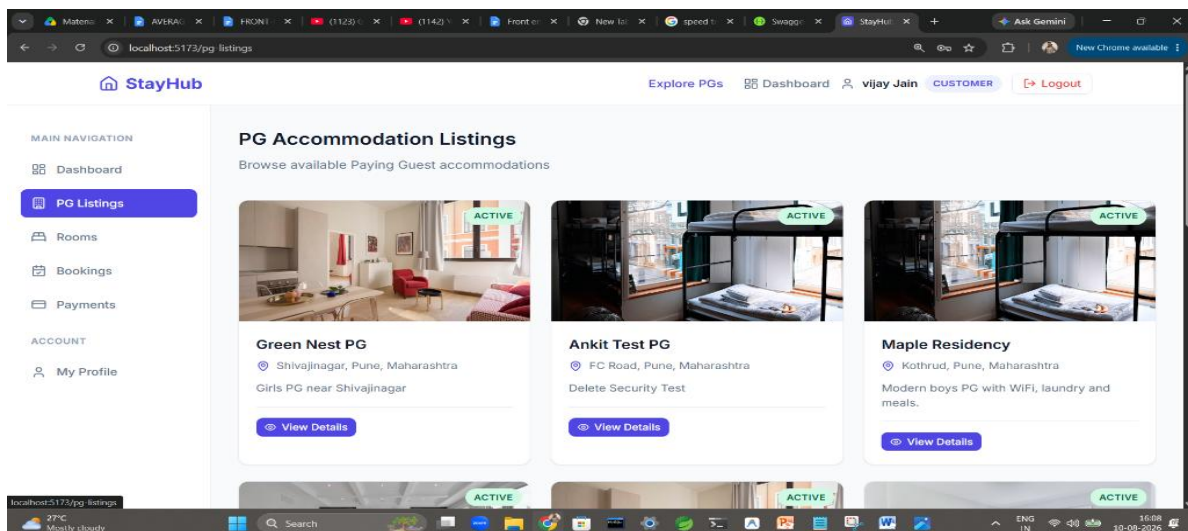


Figure 5.2: Detailed PG Property Card displaying room sharing configurations, pricing rates, and available bed inventory.

5.3 Bookings Status Filter Tabs & Audit

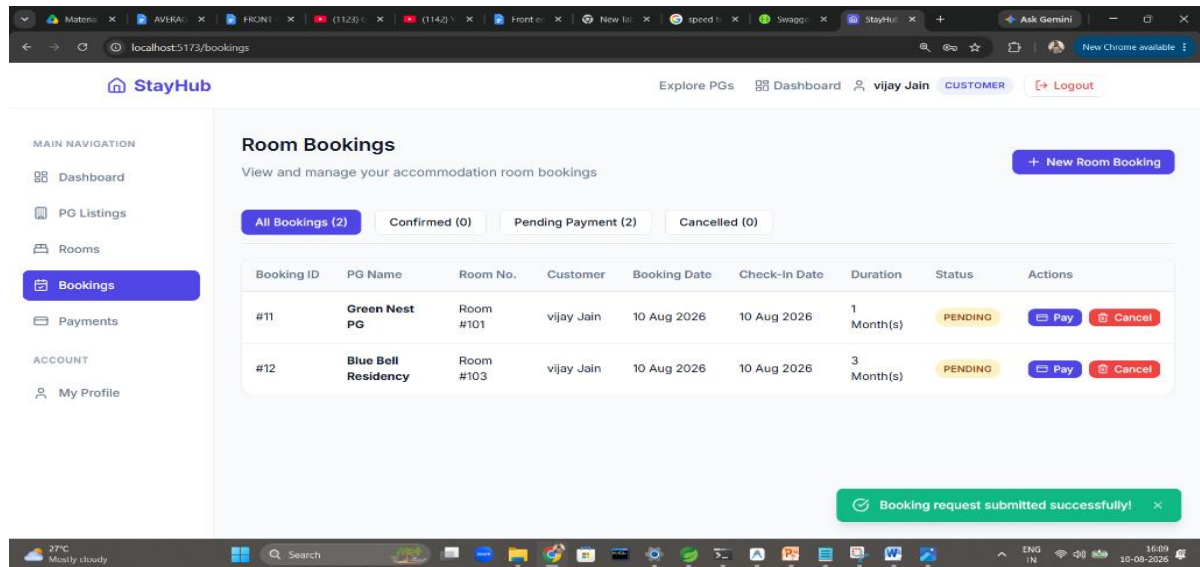


Figure 5.3: Customer Bookings Management interface featuring quick status filter tabs (All Bookings, Confirmed, Pending Payment, Cancelled).

5.4 Categorized User Management & Password Reset

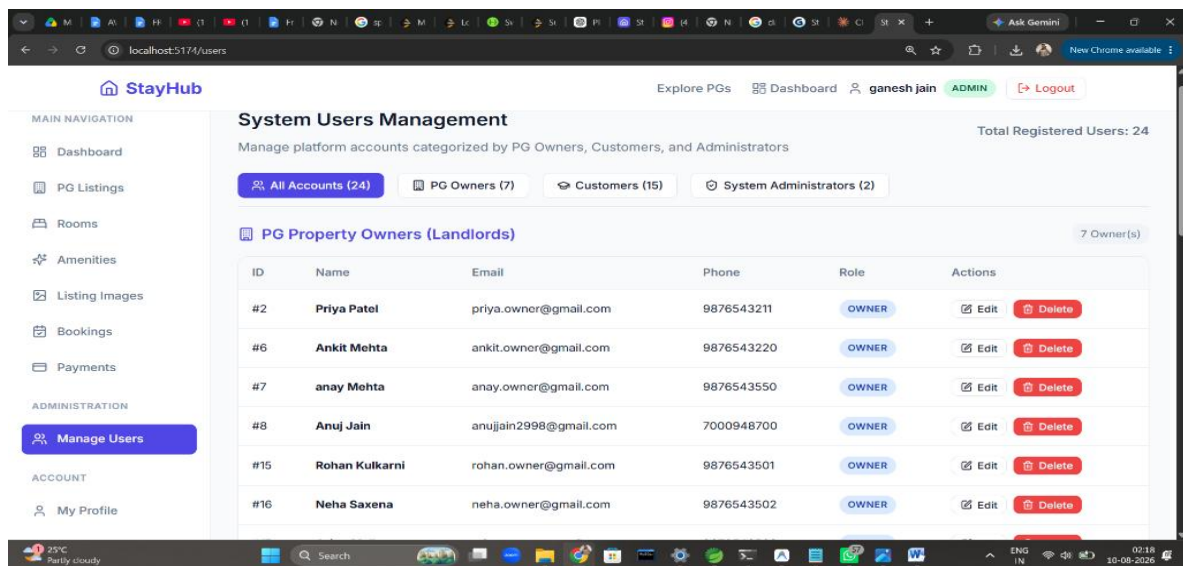


Figure 5.4: Administrator User Management interface categorized into PG Owners, Customers, and System Admins tabs.

5.5 Amenity Preset Chips & Interactive Badges Manager

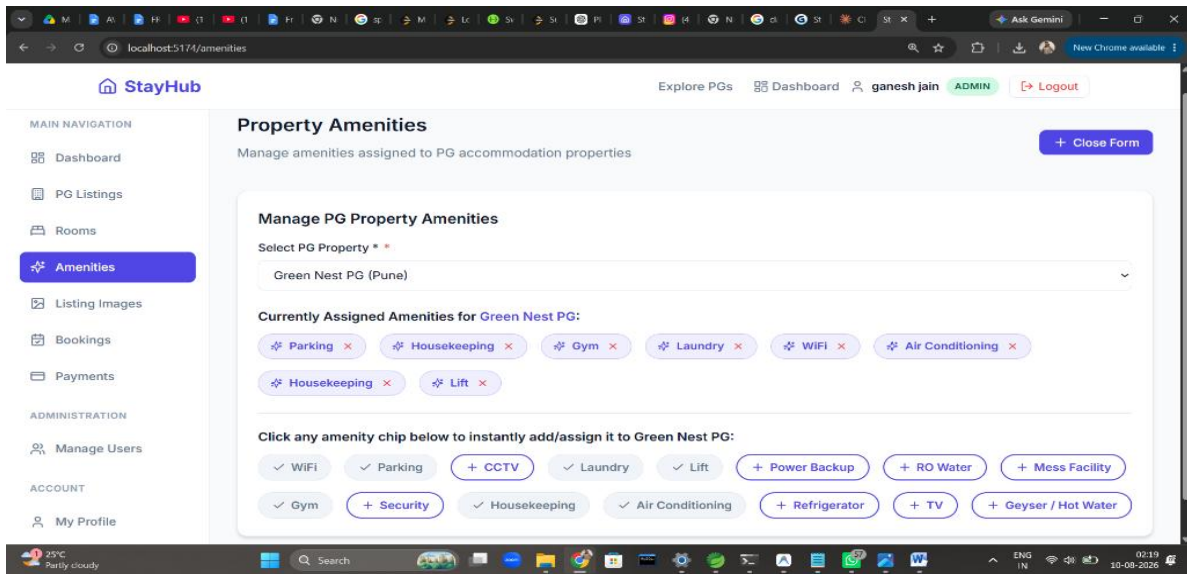


Figure 5.5: PG Property Amenity Badge Manager with 1-click preset chips and instant removal badges.

5.6 Razorpay Payment Processing & Verification

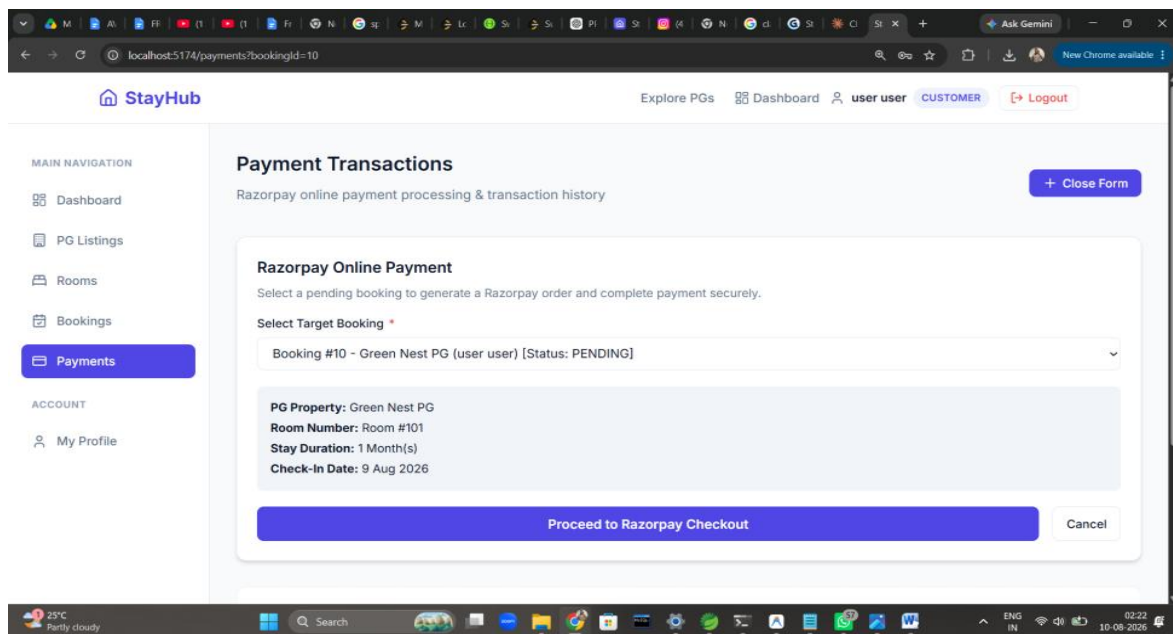
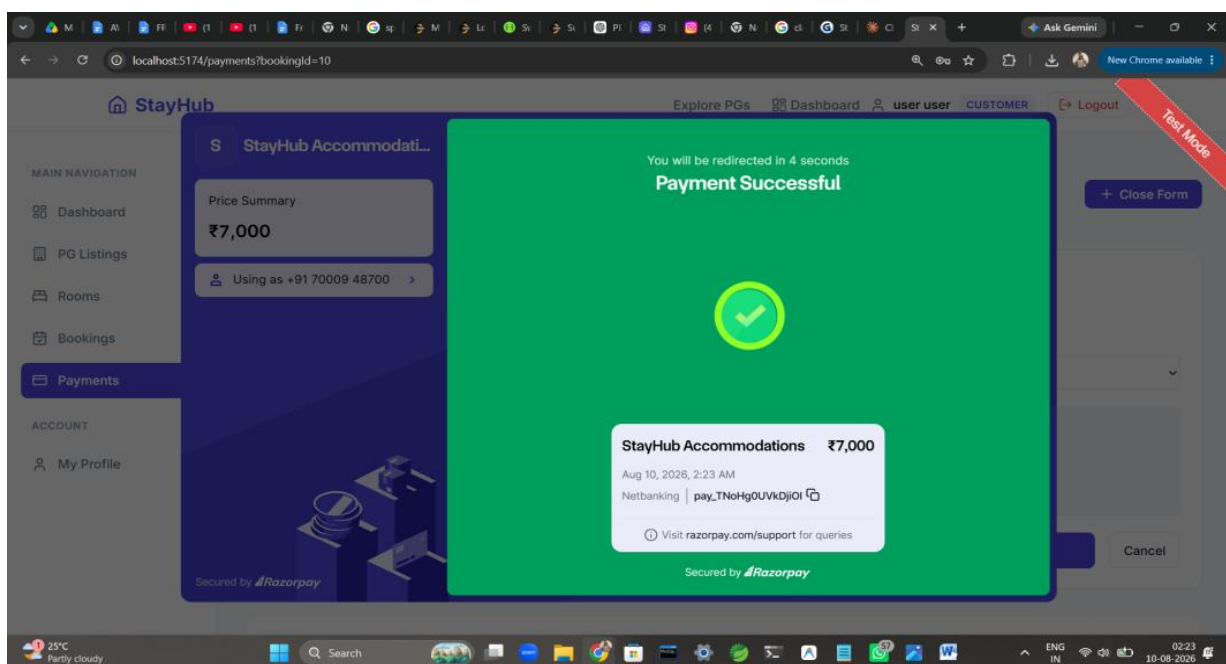
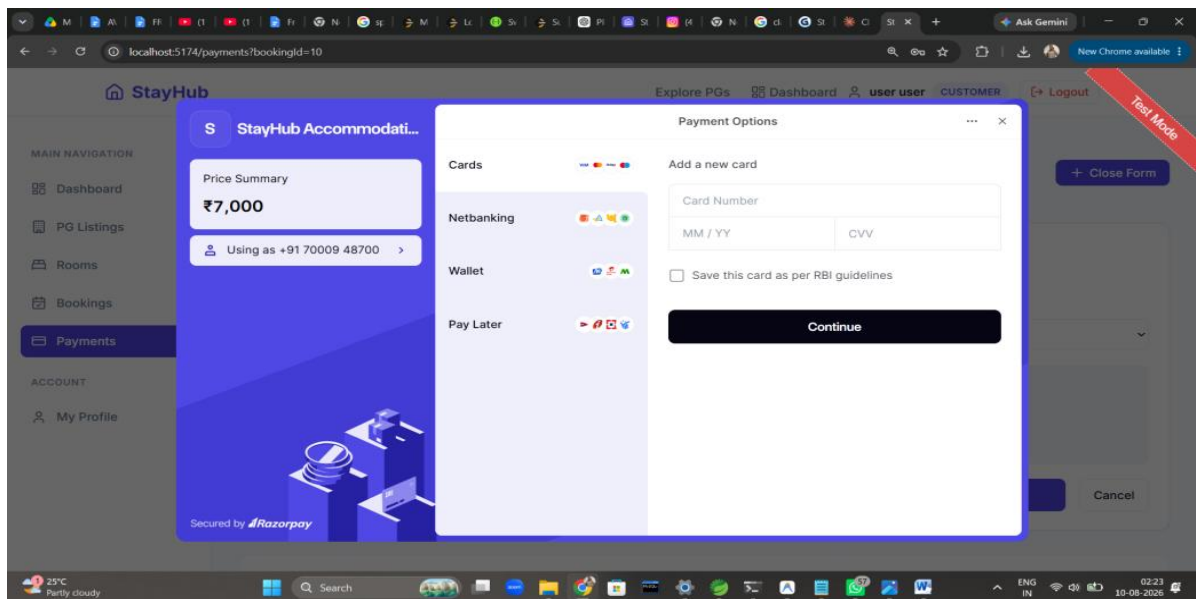


Figure 5.6: Razorpay payment order audit logs and signature verification status display.



6. CONCLUSION & FUTURE SCOPE

In conclusion, StayHub - PG Accommodation Booking and Management System addresses the key operational and technological challenges faced by modern hostel management and real estate booking portals. By combining a modern React 18 single-page application frontend with a robust Java 21 Spring Boot 3.5 RESTful micro-architecture, StayHub delivers a scalable, highly performant, and secure platform for property owners, tenants, and administrators.

Key technical milestones achieved include: (1) Real-time bed inventory synchronization and automated bed decrementation/restoration; (2) Role-Based Access Control (RBAC) powered by Spring Security and JWT tokens; (3) Seamless Razorpay payment gateway integration with HMAC SHA-256 signature verification; (4) Transactional cascading deletion (@Transactional) eliminating foreign key constraint failures; (5) Soft-delete booking cancellations (status = CANCELLED) preserving business audit trails; and (6) Modern glassmorphism UI styling with categorized user management tabs and interactive 1-click amenity chips.

Future Scope of StayHub

- **AI-Based PG Recommendation System** to provide personalized accommodation suggestions based on user preferences and behaviour.
- **Location-Based Services Integration** using maps for distance calculation, nearby facilities, and route navigation.
- **Dedicated Mobile Application Development** for Android and iOS with real-time booking and notification features.
- **Advanced Analytics Dashboard** for owners and admins to monitor occupancy, revenue, and booking trends.
- **Real-Time Notification System** using WebSockets/Firebase for instant booking, payment, and maintenance updates.
- **Digital KYC and Verification System** for secure tenant and property owner identity validation.
- **Online Complaint and Maintenance Management Module** for efficient issue tracking and resolution.

- **Digital Rental Agreement Management** with e-signatures and automated renewal reminders.
- **Enhanced Security Features** including MFA, OAuth integration, encryption, and fraud detection.
- **IoT-Based Smart Hostel Management** with smart locks, energy monitoring, and automated access control.
- **Multi-City and Multi-Property Support** to expand StayHub into a large-scale accommodation marketplace.

7. REFERENCES

1. **Spring Security Framework Reference Manual (v6.x / Spring Boot 3.x):** VMware Tanzu Documentation, 2024. Available online: <https://docs.spring.io/spring-security/reference/>
2. **Spring Boot 3.5 Reference Documentation:** VMware Tanzu Documentation, 2024. Available online: <https://docs.spring.io/spring-boot/index.html/>
3. **React 18 Official Developer Reference Documentation:** Meta Open Source, 2024. Available online: <https://react.dev/reference/react>
4. **Database Systems: The Complete Book (2nd Edition):** Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). Prentice Hall. ISBN: 978-0131873254.
5. **A Relational Model of Data for Large Shared Data Banks:** Codd, E. F. (1970). Communications of the ACM, 13(6), 377-387. <https://doi.org/10.1145/362384.362685>
6. **Patterns of Enterprise Application Architecture:** Fowler, M. (2003). Addison-Wesley Professional. ISBN: 978-0321127426.
7. **Domain-Driven Design: Tackling Complexity in the Heart of Software:** Evans, E. (2003). Addison-Wesley Professional. ISBN: 978-0321125217.
8. **Razorpay Payment Gateway API Specs & Integration Guide:** Razorpay Software Private Limited, 2024. Available online: <https://razorpay.com/docs/api/>
9. **OpenAPI 3.0 Specification & Swagger UI Standards:** SmartBear Software, 2024. Available online: <https://swagger.io/specification/>
10. **JSON Web Token (JWT) Architecture (RFC 7519):** Internet Engineering Task Force (IETF), 2015. Available online: <https://datatracker.ietf.org/doc/html/rfc7519>
11. **Hibernate ORM 6.x Developer Guide & JPA Persistence Specs:** Red Hat Middleware Documentation, 2024. Available online: <https://hibernate.org/orm/documentation/>
12. **Clean Architecture: A Craftsman's Guide to Software Structure and Design:** Martin, R. C. (2017). Prentice Hall. ISBN: 978-0134494166.